



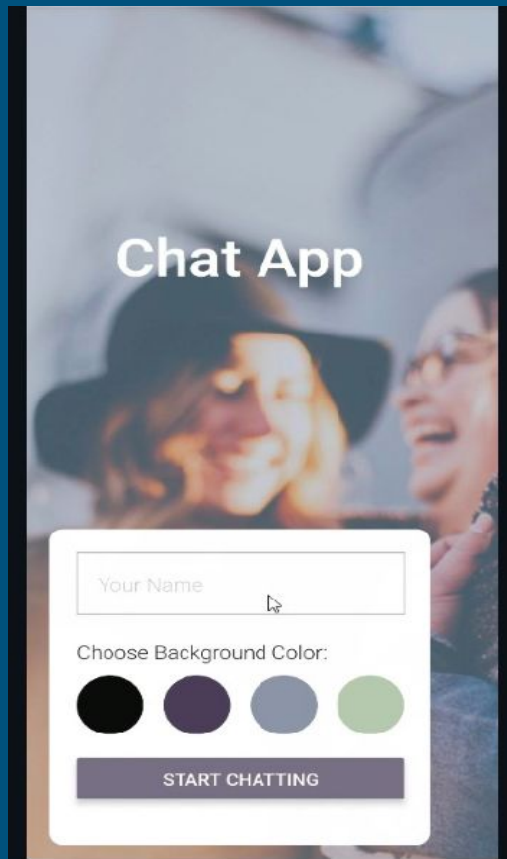
React Native Chat App Case Study

Bhavani Penugonda



Overview

Chat App is a cross-platform mobile app built with **React Native** and **Expo**, with **Firestore**, **Authentication**, and **Cloud Storage**. It provides a clean, easy-to-use interface where users can chat in real time, share images, and send their location. The app keeps messages and media synced and accessible even when offline.



Purpose and Context

This chat app was developed as a personal project during my Full-Stack Web Development course with **CareerFoundry**, with the goal of learning and applying **React Native** for mobile development. It was my first experience building a native mobile application and also served as an introduction to the **Expo CLI**. The project helped me gain hands-on experience with cross-platform development and integrating Firebase services in a real-world scenario.

Objective

To develop a fully functional chat application for mobile platforms (iOS and Android) using a single React Native codebase. The app allows users to:

- Enter a chat room with a personalized name and background color
- Send/receive text messages, images, and location data
- Use the app offline
- Ensure accessibility for users with visual impairments

Duration & Tools

Project Duration: 3 weeks

Course: Full-Stack Web Development Program, CareerFoundry

Tutor: Zaheer Abbas

Mentor : Sonia Rose Karungi

Tools & Technologies Used:

- React Native
- Expo CLI
- Firebase (Firestore, Authentication, Cloud Storage)
- Gifted Chat
- Async Storage
- Google Maps & Location API

Approach

Setting Up Expo

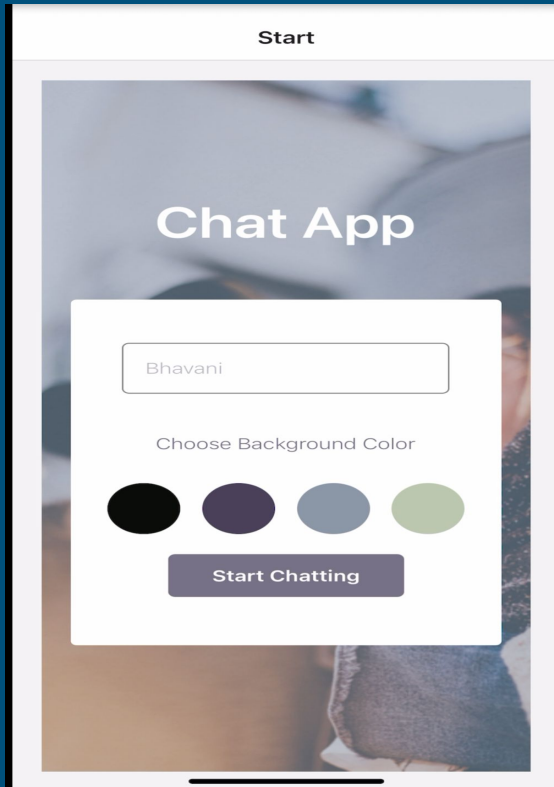
- I began by setting up my development environment, starting with tools, such as **Node.js** and the **terminal**, which I had used in earlier projects.
- I installed the **Expo CLI**, which simplified running and managing the app across both iOS and Android without needing native build tools.
- To test the app live during development, I downloaded the **Expo Go app** on my mobile device. This allowed me to scan a QR code and instantly preview the app on my phone.
- I created an **Expo account** to manage my projects and access them from any device. After setting up the account, I created a new project called **ChatApp** using the Expo CLI.
-
- This setup step was essential—it helped me gain confidence with React Native and allowed me to focus more on building features rather than managing build environments.

Approach

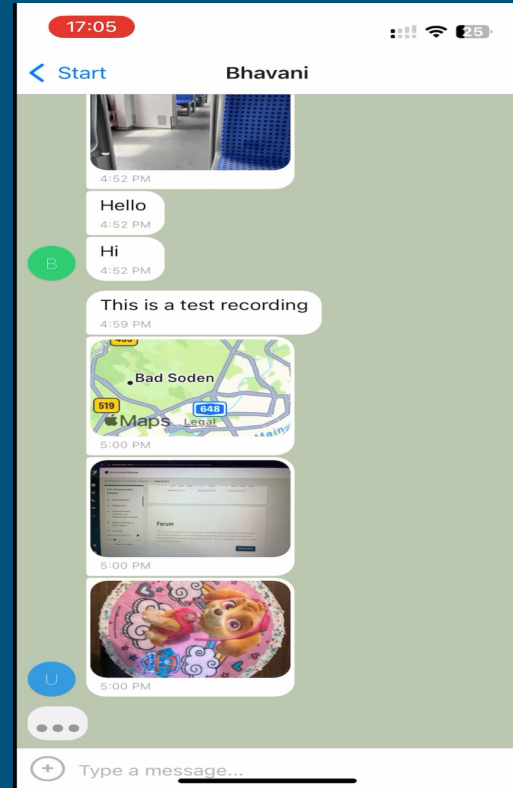
Building the User Interface

- I began with the **Start Screen**, where users can enter their name and choose a background color. This was the first interactive UI I built, and it helped me get comfortable with React Native components like `TextInput`, `TouchableOpacity`, and `View`.
- Used React Native's **ImageBackground** component to add a full-screen background image to the Start Screen, enhancing the visual design.
- Applied all styling using **StyleSheet**, including the specified **heights**, **widths**, and layout properties to match the design specifications accurately.
- I used **React Navigation** to move from the **Start Screen** to the **Chat Screen**. Created a "Start Chatting" button on the Start Screen that triggers navigation when pressed.
- I retrieved the user's name and background color on the Chat Screen by accessing the values passed from the Start Screen.
- I set the navigation bar title to show the user's name, making the chat screen feel more personalized.
- I used the selected background color from the Start Screen to style the Chat Screen, so each user sees their preferred color when chatting.

Start Screen



Chat screen



Approach

Chat UIs & Accessibility

- To implement the chat interface, I used the **Gifted Chat** library, a powerful and customizable open-source React Native component designed specifically for building chat UIs.
- Gifted Chat provided out-of-the-box features such as
 - message Bubbles with Timestamps
 - user Avatars
 - Input Field with Typing Indicator
 - support for sending texts, images and custom actions
- I used **Gifted Chat's renderActions** function to add custom buttons that allow users to choose images, take photos, or share their location.
- The library handled rendering messages efficiently in a scrollable view, which made it easier to manage large conversations.

Approach

- React Native provides built-in **accessibility support**, allowing developers to make mobile apps more usable for individuals with visual or physical impairments.
- I used **accessibility props like accessibilityLabel, accessibilityHint, accessibilityRole** to ensure that key elements like buttons, input fields, and color selectors were **navigable and understandable** for users relying on assistive technologies.
- Prioritizing accessibility allowed me to build a more inclusive and user-friendly app experience.

Approach

Data Storage

- I chose **Firebase** for its robust backend services, including real-time database, authentication, and cloud storage, which fit perfectly with the chat app's requirements.
- First, I set up a Firebase project in the Firebase Console and configured Firestore to store chat messages securely and in real time.
- I enabled **anonymous authentication** to allow users to join the chat quickly without requiring sign-ups or passwords, streamlining the user experience.
- Integrated Firebase Authentication in the app to manage user sessions and ensure each message could be linked to a unique user ID.
- Connected the app with **Cloud Firestore** to read and write messages, syncing them instantly across devices. This ensured that conversations stayed updated in real time.
- Implemented **Firebase Cloud Storage** to upload and store user images and photos taken within the app, making media accessible and sharable in chats.

Approach

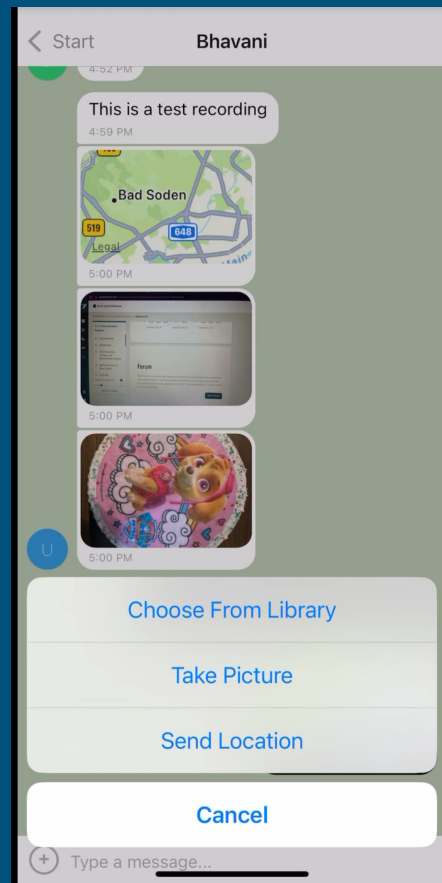
Storing Data on the Client Side

- Used AsyncStorage to provide users with offline access to their sent and received messages, ensuring chat history is always available.
- I used **NetInfo**, an Expo-recommended library, to detect whether the user is online or offline so the app could decide to load data from Firebase or AsyncStorage accordingly.
- Then, I used the useNetInfo hook, which gives me the current connection status and updates automatically whenever the network changes.
- This way, the app knows in real time if the user is offline and can load saved messages from AsyncStorage.
- And then for testing the app offline, used the simulator's network settings to disable internet and confirmed that previously loaded messages were still accessible from AsyncStorage while offline.

Approach

Communication, Tracking, and Measurement Features

- I used Gifted Chat renderActions to add a custom options menu in the chat input toolbar, allowing users to easily select an image from their library, **Take Picture**, **Choose From Library**, or **Send Location**—all accessible from a single menu.
- To make this work, I integrated **Expo's ImagePicker**, which allowed users to either pick images from their device or capture new ones using the camera and send them instantly in the chat.
- I used **Expo's Location API** to request location permissions and retrieve the user's **real-time coordinates**, which I sent as part of the chat message.
- On the chat screen, I used a **MapView** component to display the shared location, providing users with a visual preview of the sender's position directly in the chat.
- I integrated **Firebase Storage** to securely upload and store images sent by users in the chat, using Expo's **ImagePicker** to let users select or capture images that were then uploaded directly to Firebase Storage.
- After a successful upload, I retrieved the image URL and then used Gifted Chat's method to send the image as a message, allowing it to render directly in the chat.



Retrospective

What went well?

I set up navigation between screens and passed props like username and background color smoothly. This helped personalize the UI and made the app feel more user-friendly. I was able to successfully build a real-time chat app using **React Native**, **Gifted Chat**, and **Firebase**, even though it was my first time combining all these technologies.

What didn't go well?

I faced several challenges, particularly when integrating Firebase with custom features like image uploads, location sharing, and real-time updates. One of the hardest parts was making sure these features worked smoothly within Gifted Chat while keeping the user experience seamless. Getting everything to sync properly with Firebase, especially image uploads and message rendering, took some trial and error.

Future Steps

In the future, I plan to add the ability to send **audio files** and **PDF documents**, expanding the app's functionality to support more types of media sharing.

Final Thoughts

Building this chat app was a great learning experience. I learned a lot about using React Native and Firebase, especially with features like image and location sharing. There were some challenges, like handling uploads and permissions, but I worked through them. I'm excited to add more features like sending audio files and documents to make the app better.

[View the code on GitHub](#)